STACKLY TOURISM

Go beyond *the familiar.*

A technical reference for the Stackly Tourism public website, authentication flow, and workspace prototype.

PROJECT
Stackly Tourism — adventure travel website and role-based workspace prototype

STACK
HTML5, CSS3, vanilla JavaScript, SVG, localStorage

HOSTING
GitHub Pages — static deployment from the main branch

AUDIENCE
Developers, designers, maintainers, and project reviewers

CONTENTS

What this document covers

The pages are ordered as a working reference: first the project context and design system, then the page-by-page implementation, followed by responsive behaviour, deployment, and the closing assessment.

3	Introduction	3
4	Technology stack and architecture	4
5	Colour palette and typography	5
6	Project structure	6
7	Shared layout, navigation, 404 and route guard	7
8	Home page	8
9	About page	9
10	Services page	10
11	Journeys page	11
12	Planning page	12
13	Contact page	13
14	Authentication and session handling	14
15	Client dashboard	15
16	Consultant dashboard	16
17	Responsive behaviour	17
18	Deployment and conclusion	18

How to read it

Section numbers in the page headers match the contents list. File names and code tokens are set in monospace. Tables are used where a list would hide the relationship between files, behaviours, and breakpoints.

> **Build note.** The site has no build step, package manager, framework, or server-side runtime. Every file in the repository is delivered to the browser as written.

Document scope

Includes	Public pages, shared components, auth prototype, both dashboards, responsive rules, GitHub Pages deployment.
Excludes	Real booking inventory, payment processing, server-side authentication, email delivery, database operations.
Source of truth	The repository itself. This document describes the code as it is organised and presented in the browser.

Document version 1.0.

INTRODUCTION

A quiet site with a working prototype underneath

Stackly Tourism is a concept website for small-group journeys. The public pages establish the tone and the route-planning story; the authentication screens and two dashboards show how a client or consultant would move into a working workspace.

Project overview

The site is a static, multi-page build. Each public page is a complete HTML document with its own CSS layer and page-specific JavaScript. Shared behaviour lives in `js/main.js` and `js/session.js`. There is no application server: the browser requests the files directly from GitHub Pages, and the workspace prototype stores its state in `localStorage`.

The visual direction is editorial rather than transactional. Large serif display type, warm paper surfaces, muted photography, and restrained gold accents carry the public pages. The client dashboard keeps the same type and colour language in a dark workspace shell. The consultant dashboard uses the same shell with a teal role accent.

Objectives

- Present the journey-planning concept with a clear, calm visual hierarchy.
- Keep every page usable on a 320px viewport without losing the editorial feel.
- Make the prototype feel complete: navigation, forms, validation, success states, role selection, and dashboards.
- Deploy the whole site as a static project under a GitHub Pages subpath.
- Keep the code readable enough for another developer to pick up without a framework.

11	10	14
HTML pages	CSS files	JavaScript files

What is included

Public site	Home, About, Services, Journeys, Planning, Contact
Account flow	Login, sign-up, role selection, session persistence
Workspaces	Client dashboard, consultant dashboard, settings, charts, messages
System page	Custom 404 page and guarded content links
Assets	73 WebP images, 10 SVG fallbacks and map artwork, two logo files

Boundaries

Forms simulate submission with short timeouts. Authentication accepts any valid email and a password that meets the visible policy; there is no credential store. Dashboard data is deterministic mock data. No booking, payment, email, or database action leaves the browser.

REVIEW NOTE
Treat the dashboards as interaction prototypes. The UI states, validation rules, and transitions are real; the data and account actions are not connected to a backend.

TECHNOLOGY STACK AND ARCHITECTURE

Static files, shared tokens, browser-side state

The architecture is deliberately small. HTML carries the content, CSS carries the design system, and a set of classic-script modules add navigation, motion, validation, charts, and the workspace shell.

1. Request
Browser requests a page from the GitHub Pages project URL.

2. Deliver
Pages, styles, scripts, and local images arrive as static files.

3. Enhance
Deferred scripts add navigation, reveals, forms, charts, and dashboards.

4. Persist
Session and preferences are kept in localStorage on the device.

Stack at a glance

Layer	Implementation
Markup	HTML5 documents, semantic sections, ARIA labels, inline SVG icons.
Styling	CSS3 custom properties, Grid, Flexbox, clamp() type scales, media queries, reduced-motion rules.
Behaviour	Vanilla JavaScript in classic-script IIFEs. No framework, bundler, or transpiler.
Charts	Hand-built SVG in `js/charts.js`; no chart library.
State	localStorage for the session, client preferences, consultant preferences, and remembered login.
Fonts	Google Fonts: Instrument Serif and Inter, with local system fallbacks.
Hosting	GitHub Pages, deployed from the repository's `main` branch at the project root.

Script loading

Public pages load `main.js` with `defer`, followed by a page-specific script. The dashboards load their data module, chart module, and workspace script in that order. The client dashboard loads `session.js` before the workspace shell so the role guard can run before the view is built.

Shared and page-specific code

File	Responsibility
`js/main.js`	Navigation, scroll effects, reveals, counters, carousel, FAQ, newsletter, route guard.
`js/session.js`	Session shape, validation, role guard, initials, avatar colour.
`js/auth.js`	Login and sign-up validation, role selection, success states, redirects.
`js/contact.js`	Contact field validation, summary, counter, simulated submission.
`js/dashboard.js`	Client workspace views, settings forms, messages, charts mounting.
`js/consultant-dashboard.js`	Consultant workspace views, client scope, rate card, inbox, settings.
`js/charts.js`	Shared SVG line, bar, radar, and heatmap chart builders.

> **Design choice.** Shared visual tokens live in `css/style.css`; page layers extend them instead of redefining the palette. The dashboards add a dark surface scale and chart colours, while the consultant theme overrides only the accent family.

Browser APIs in use

IntersectionObserver for reveals and counters, matchMedia for responsive and reduced-motion checks, ResizeObserver for the 404 route drawing, URLSearchParams for the login `next` parameter, Blob and object URLs for CSV export, and the native `dialog` element for the about-page lightbox.

COLOUR PALETTE AND TYPOGRAPHY

Warm paper, dark ink, restrained gold

The palette is intentionally narrow. Most surfaces are paper or cream; dark sections use near-black ink; gold is reserved for emphasis, active states, and small rules.

Core palette

Swatch	Token	Hex	Use
	`--ink`	`#0B0B0F`	Dark sections, body text, dashboard shell
	`--bone`	`#F4F1EA`	Primary light surface, dark-section text
	`--paper`	`#EEEAE1`	Alternating sections, table headers
	`--cream`	`#E8E3D8`	Callouts, soft panels, form surfaces
	`--accent`	`#C8A15A`	Gold accents, active states, rules
	`--gold-deep`	`#765B2D`	Accent text on light surfaces
	`--muted`	`#625D53`	Secondary copy, captions
	`--soft`	`#8A8578`	Labels, metadata, quiet UI

Workspace accents

Area	Accent	Use
Client dashboard	`#C8A15A` / `#D7B574`	Gold primary actions, chart series, focus ring
Consultant dashboard	`#7CC0B1` / `#9AD6C8`	Teal role theme for active navigation and primary actions
Status	`#A8D3B6` / `#E5C98F` / `#F0A396`	Success, warning, danger states in the workspace
Chart series	`#C8A15A`, `#83A99B`, `#A49AC8`, `#D18E6C`	Line, bar, radar, and heatmap data

Typography

Role	Family	Scale / use
Display	Instrument Serif	`clamp(4.5rem, 10.1vw, 10rem)`
Section title	Instrument Serif	`clamp(2rem, 1.55rem + 2vw, 3.5rem)`
Body	Inter	`clamp(.95rem, .9rem + .2vw, 1.0625rem)`
Small / UI	Inter	`.7rem – .94rem`
Mono	SFMono / Consolas	File paths, numbers, labels

TYPE RULE

Display type is set tight and large; body copy stays open. The italic serif is used for emphasis words, not for whole paragraphs. Uppercase labels are small and tracked, never used for long copy.

Spacing and shape

Spacing follows a 4px base scale: 4, 8, 12, 16, 24, 32, 48, 64, 96, 128. Radii are deliberately small: 2px for buttons and fields, 8px for panels, 16px for larger surfaces, and a pill only for tags and toggles. Borders are hairlines, usually 8–14% ink or bone.

Motion

Reveals, counters, carousels, and the 404 route use long easing curves. All non-essential motion is disabled under `prefers-reduced-motion: reduce`. The dashboards keep transitions short so table and form work stays responsive.

> **Contrast note.** Gold is never used as body text on paper. It is used for accents, rules, and short labels. Secondary copy uses the muted brown-grey rather than a lighter gold.

PROJECT STRUCTURE

One root, three working folders

The repository is flat at the top level and grouped by asset type. HTML sits at the root so GitHub Pages can serve each page directly; CSS, JavaScript, and images live in their own folders.

Folder tree

```
Stackly Adventure tourism/
├─ index.html
├─ about.html
├─ services.html
├─ journeys.html
├─ planning.html
├─ contact.html
├─ login.html
├─ signup.html
├─ client-dashboard.html
├─ consultant-dashboard.html
├─ 404.html
├─ css/
│  ├─ style.css
│  ├─ about.css
│  ├─ services.css
│  ├─ journeys.css
│  ├─ planning.css
│  ├─ contact.css
│  ├─ auth.css
│  ├─ dashboard.css
│  ├─ consultant-dashboard.css
│  └─ 404.css
├─ js/
│  ├─ main.js
│  ├─ session.js
│  ├─ auth.js
│  ├─ about.js
│  ├─ services.js
│  ├─ journeys.js
│  ├─ planning.js
│  ├─ contact.js
│  ├─ dashboard.js
│  ├─ consultant-dashboard.js
│  ├─ charts.js
│  ├─ client-data.js
│  ├─ consultant-data.js
│  └─ 404.js
└─ assets/
   ├─ logo/
   └─ img/
      ├─ home/
      ├─ about/
      ├─ services/
      ├─ journeys/
      ├─ planning/
      ├─ contact/
      ├─ login/
      ├─ signup/
      └─ *.svg fallbacks
```

Inventory

Type	Count	Notes
HTML	11	10 public/auth/workspace pages plus 404
CSS	10	One shared system, one layer per page group
JavaScript	14	Shared modules, page modules, mock data
WebP	73	Page-specific photography
SVG	10	Fallbacks, map artwork, brand marks
Logo	2	Light and dark WebP wordmarks

Organisation rules

- Page CSS is named after the page: `services.css`, `planning.css`, and so on.
- Page JavaScript is named after the page and loaded only where needed.
- Shared tokens, navigation, footer, and buttons stay in `style.css`.
- Images are grouped by the page that uses them, with a small set of shared fallback SVGs.
- No build output, generated bundles, or hidden dependencies live in the repository.

Why the root is flat

GitHub Pages serves a project site under a repository subpath. Keeping the HTML at the root and using relative links means the same build works at `/` locally and at `/Stackly-Adventure-Tourism/` on Pages. There is no server rewrite layer to hide a broken path.

MAINTENANCE NOTE

If a page is added, give it a matching CSS and JS file only when it needs unique behaviour. Shared code should stay shared; page layers should not duplicate tokens.

SHARED LAYOUT, NAVIGATION, 404 AND ROUTE GUARD

The parts that stay the same

Every public page uses the same fixed header, the same footer structure, and the same shared script. This is where navigation, the 404 page, and the link guard are defined.

Header and navigation

The header is fixed and becomes a dark translucent bar after 40px of scroll. The desktop layout places the brand left, the primary links in the centre, and the "Plan a journey" call to action right. Below 900px the links move into a full-screen panel opened by the menu button.

Element	Behaviour
Brand	Links to `index.html`; uses the light or dark wordmark by surface.
Primary links	Home, About, Services, Journeys, Planning, Contact.
Nav CTA	Links to `login.html`.
Mobile panel	Full-screen, focus-managed, closes on Escape, link click, backdrop click, or desktop resize.
Current page	Marked with `aria-current="page"` and a gold underline.

Footer

The footer repeats the brand statement, a social row, three link columns, a newsletter form, and legal links. Social links are circular SVG icon buttons for Instagram, Facebook, X, LinkedIn, YouTube, and Pinterest. The newsletter is a front-end demo: it validates the email, shows a success message for three seconds, then returns to its default note.

404 page

`404.html` is a styled error page with a drawn route, a "Go home" action, and a "Go back" action. If there is no usable history, "Go back" falls back to `index.html`. The page is deliberately excluded from the route guard so its own links keep working.

Route guard

`main.js` rewrites non-allowed content links to `404.html` and intercepts clicks in the capture phase. The guard exists because the concept site intentionally leaves most content calls to action unbuilt. Header, footer, dashboard sidebar, forms, testimonial controls, FAQ, and the skip link remain usable.

Allowed	Redirected
Header and footer links	Hero and section CTAs
Dashboard sidebar links	Service cards and story links
Forms and form buttons	"View work" and similar content links
Testimonial carousel controls	Principle and team links outside the footer
FAQ sections and skip link	Any other content anchor or button

EXEMPT PAGES
Auth pages, dashboard pages, and the 404 page do not run the guard. That keeps login, sign-up, workspace navigation, and the error page functional.

Shared script responsibilities

- Sticky header state and mobile navigation.
- Scroll-linked hero drift, device tilt, timeline progress, and CTA scale.
- Reveal observer, counters, image fallbacks, and spotlight hover.
- Testimonial carousel, FAQ accordion, newsletter validation, and current year.
- Link guard and 404 fallback behaviour.

HOME PAGE

Go beyond the familiar.

index.html
css/style.css
js/main.js

The home page is the longest public page. It introduces the brand, the travel philosophy, the service families, the workspace preview, the operating targets, the process, guest-note placeholders, and the FAQ.

Section order

Section	Content and role
Hero	Large serif statement, route image, two floating stat cards, scroll cue.
Story	Sticky-style editorial intro beside a two-image gallery; field-note aside.
Services	Six-card bento grid linking to service anchors.
Dashboard preview	Mock field-journal device with map, weather, checklist, and chart.
Metrics	Four operating targets with animated counters.
Process	Four-step timeline with a scroll-linked rail.
Testimonials	Three guest-note placeholders in a scroll-snap carousel.
FAQ	Five questions in native details/summary elements.
Final CTA	Large closing statement, next-step list, contact link.

Behaviour

- Hero image parallax and the two stat cards use a scroll-linked drift variable.
- Service cards use pointer-tracked spotlight and magnet offsets.
- The dashboard device tilts with scroll progress; its checklist and chart animate when visible.
- Counters run once when they enter the viewport; reduced-motion users see final values immediately.
- The carousel supports arrows, dots, keyboard arrows, drag, and pause on hover or focus.
- The FAQ uses native details; opening one closes the others.

Responsive notes

Below 767px the hero becomes a single column. At 520px and below the floating stat cards leave the image and stack beneath it; the visual becomes full width, the image ratio changes to 1.15, and the visual index is hidden. The story aside sits in the empty left gallery cell rather than on top of the small caption.

BUILD NOTE
The home page has the highest density of motion on the site. Keep an eye on scroll performance when adding sections; the shared script already limits work to one animation frame per scroll event.

ABOUT PAGE

The people behind the route.

about.html
css/about.css
js/about.js

The About page explains how Stackly works: local leadership, small groups, slower miles, and room for weather. It mixes editorial copy with milestones, principles, team profiles, a culture gallery, and operating numbers.

Section order

Section	Content and role
Hero	Two-image collage, large title, short introduction, journey link.
Manifesto	Wide image with a statement about listening before planning.
Milestones	Three year buttons that change the story panel.
Principles	Four principle cards with image previews on hover and focus.
Team	Four profile cards plus a join-the-team card.
Culture	Image gallery with a native dialog lightbox.
Numbers	Operating targets and a closing note.
Closing	Contact and careers links.

Interactions

- Milestone buttons switch the active year and update the panel content.
- Principle cards update a shared preview image on hover and keyboard focus.
- Gallery items open a dialog with previous/next controls and a caption.
- Counters animate once when the numbers section enters the viewport.

Implementation notes

`about.js` owns the milestone state, principle preview, lightbox, counters, and image fallbacks. The lightbox uses the native `dialog` element and restores focus when closed. The gallery buttons carry `data-full` paths, so the full-size image is loaded only when needed.

The page uses the same reveal observer as the home page but with shorter distances. Images are lazy-loaded below the hero; the hero image is the only high-priority asset.

Responsive notes

The hero collage collapses from two columns to one. Milestone buttons become a horizontal scroll strip on narrow screens. Principle previews are hidden below the point where hover is reliable; the cards remain readable as text. The lightbox fills the viewport and stacks its controls on phones.

> **REVIEW NOTE**
> The team cards are concept profiles, not real staff records. The join card links to the contact page and is one of the few content links allowed through the route guard because it sits in the page body? It does not: it is redirected by the guard. Use the footer contact link to reach the form.

SERVICES PAGE

What we do out there.

services.html
css/services.css
js/services.js

The Services page is the most component-heavy public page. It combines a service index, discipline cards, detailed service rows, a diagonal process path, a toolkit marquee, proof cards, pricing models, and a FAQ.

Section order

Section	Content and role
Hero	Split layout with a service image and the opening statement.
Service index	Six rows with mobile tags, previews, and active state.
Disciplines	Six capability cards with image backgrounds.
Service details	Three detailed rows with deliverables and expanders.
Process	Four steps on a diagonal path; vertical timeline on mobile.
Toolkit	Dual marquee of working methods.
Proof	Three outcome cards.
Models	Three engagement models with a monthly/quarterly toggle.
FAQ and CTA	Four questions and a closing conversation link.

Process timeline

On desktop the four steps sit on a diagonal SVG path with a scroll-linked draw. Below 900px the path is hidden and the steps become a vertical timeline. The number circle sits in a 44px left gutter with a 12px gap before the heading; the rail is centred through the circles.

Interactions

- Service index rows switch the active preview and update the pressed state.
- Pricing toggle changes the displayed period and price labels.
- FAQ uses native details; the process nodes activate as the timeline scrolls.
- Images use local WebP assets with SVG fallbacks.

Responsive notes

The hero, service index, disciplines, details, models, and FAQ all collapse to single-column layouts. The process timeline is the key mobile transformation: the diagonal drawing is replaced by a vertical rail, and the number circles move into the left gutter. The services page is the best place to check that a layout change does not reintroduce text overlap at 320px.

BUILD NOTE
The process circle spacing is intentional: `padding-left: 44px` and `left: -44px` keep the 32px circle clear of the heading. The rail sits at 16px so it passes through the circle centre.

JOURNEYS PAGE

Routes with room to change.

journeys.html
css/journeys.css
js/journeys.js

The Journeys page presents the journey concept as a set of editable routes rather than fixed packages. It is built around a featured route, a filterable index, traveler outcomes, and a closing conversation.

Section order

Section	Content and role
Hero	Large route image and the journey promise.
Arc	Three-part story of how a fixed itinerary becomes a route.
Featured	One highlighted journey with image, notes, and details.
Index	Six journey cards with filters and expandable details.
Outcomes	Before/after comparison with two images.
Voices	Three traveler portraits and quotes.
CTA	Conversation link and secondary question link.

Filters and cards

The index uses buttons for “All”, “First journey”, “Family”, “Alpine”, and “Private”. Each card has a details toggle that expands practical notes without leaving the page. The active filter is reflected with `aria-pressed`; the expanded state uses `aria-expanded`.

Behaviour

- Filter buttons show and hide cards by data category.
- Card toggles reveal the route notes and keep the focus in place.
- The featured image is preloaded; the rest are lazy-loaded.
- Reveal and image fallback behaviour comes from `main.js`.

Responsive notes

The journey index moves from a multi-column card grid to a single column. Filter buttons wrap into a compact strip. The featured layout stacks the copy above the image. The outcomes comparison becomes a vertical pair rather than a side-by-side overlap.

> **REVIEW NOTE**
> The journeys are concept routes, not live inventory. Dates, prices, and availability are intentionally absent from the public page. The contact page is the handoff point for a real conversation.

PLANNING PAGE

Plan the days, not just the distance.

planning.html
css/planning.css
js/planning.js

The Planning page turns the service promise into a planning method: principles, staged execution, a small scenario simulator, a quarterly roadmap, a live plan preview, and a readiness checklist.

Section order

Section	Content and role
Hero	Planning statement with a coastal image and two CTAs.
Principles	Three planning rules with short explanations.
Stages	Five expandable planning stages with detail panels.
Simulator	Conservative, balanced, and aggressive presets that change the numbers.
Roadmap	Four quarters, each with a different image and plan summary.
Live plan	Overview, capacity, and risks tabs with a snapshot image.
Readiness	Checklist and final conversation CTA.

Interactions

- Stage rows expand one panel at a time and keep the row state visible.
- Simulator presets update the numbers and the active button.
- Roadmap quarter buttons swap the image and summary.
- Live plan tabs switch panels with correct `role="tab"` and `aria-selected` states.

Responsive notes

The planning layouts are table-like on desktop and stack on narrow screens. Stage rows keep their numbers and headings readable; the simulator controls wrap into a vertical group; roadmap images sit below the copy; the live plan tabs become a horizontal scroll strip if needed.

> **BUILD NOTE**
> The page is a good example of progressive disclosure: each section gives the headline first and the detail on request. Keep that pattern when adding new planning tools; avoid opening every panel by default.

CONTACT PAGE

Bring the question. We will bring the next step.

contact.html
css/contact.css
js/contact.js

The Contact page is the main conversion surface. It combines studio information, a map card, a validated enquiry form, and a closing statement.

Section order

Section	Content and role
Hero	Large “Let’s talk” statement and a short deck.
Studio	Address, contact details, map card, social links.
Form	Name, email, company, topic, message, consent.
Closing	Image background and a final conversation line.

Form validation

Field	Rule
Name	2–60 characters; letters, spaces, hyphens, apostrophes.
Email	Valid email address; trimmed and lowercased.
Company	Optional, up to 80 characters.
Topic	Required select.
Message	20–1000 characters; live counter.
Consent	Required checkbox.

Submission behaviour

The form uses custom validation on blur and submit. Invalid fields are marked with `aria-invalid`, inline messages, and a summary at the top. A valid submission shows a “Sending…” state for 900ms, resets the form, then shows a success banner for 3 seconds before removing it.

The hidden honeypot field catches simple bots. If it is filled, the form still runs a simulated request but does not show the success banner. No data is sent anywhere.

Responsive notes

The studio and form columns stack; the map card keeps its aspect ratio; the form fields remain full width; the closing image becomes a background behind the text. At 320px the form is the main content and the map sits above it, not beside it.

BUILD NOTE
Replace the simulated timeout in `contact.js` with a real POST when a backend is available. Keep the validation, summary, and success timing; they are already written for a production handoff.

AUTHENTICATION AND SESSION HANDLING

Two forms, two roles, one local session

Login and sign-up share the same split auth shell: an image panel on the left and the form on the right. Both forms validate in the browser, then create a local session that the dashboards read.

Login

The login form asks for work email, password, and a role. It validates the email shape, the password policy, and the role choice. A valid submit shows a success banner for 3 seconds, then redirects to the selected workspace.

Sign-up

Sign-up collects full name, work email, company, role, password, confirmation, and terms consent. The password field includes a live strength meter and a five-rule checklist. On success, the form is replaced by a success card with a direct link to the chosen dashboard.

Validation rules

Rule	Detail
Email	Standard address shape; disposable domains are rejected.
Password	10–72 characters, upper and lower case, number, symbol, no spaces.
Common passwords	A short blocklist is checked before accepting the password.
Confirmation	Must match the new password exactly.
Terms	Required before account creation.

Session shape

```
{
  name: "Traveler Name",
  email: "traveler@example.com",
  company: "Company",
  role: "client" | "consultant",
  createdAt: ISO string,
  lastLogin: ISO string
}
```

Stored under `zstackly.session.v1`. The session module trims and validates the shape on read; malformed data is removed. UI preferences use `zstackly.client.prefs.v1` and `zstackly.consultant.prefs.v1`. The "remember me" choice uses `zstackly.auth.remember`.

Role guard

`session.js` exposes `requireRole()`. If there is no session, the user is sent to login with a `next` parameter. If the role does not match the dashboard, the user is sent to the correct dashboard instead. The login form only accepts a `next` value that resolves to the expected dashboard.

PROTOTYPE NOTE

There is no credential store. Any valid email plus a policy-compliant password creates a session. Replace `auth.js`'s simulated timeouts with real endpoints before treating this as production authentication.

CLIENT DASHBOARD

A calm workspace for the journey.

client-dashboard.html
css/dashboard.css
js/dashboard.js

The client workspace turns the public promise into a practical shell: a persistent sidebar, a searchable top bar, a route-driven view area, and a set of settings forms. It uses deterministic mock data and keeps UI preferences on the device.

Views

View	Content
Overview	Journey summary, upcoming milestones, actions, recent activity.
Journeys	Journey cards, status, progress, detail drawer.
Planning	Plan stages, tasks, dates, readiness signals.
Reports	Hand-built SVG charts, series toggles, CSV export.
Documents	Document list, filters, download simulation.
Messages	Thread list, conversation, composer, send state.
Settings	Profile, preferences, security, danger zone.

Settings forms

- Profile: name and company; email and role are read-only.
- Preferences: timezone, digest, email notifications, compact density.
- Security: current password, new password, confirmation; policy checks.
- Danger zone: email confirmation, simulated deletion, 3-second result.

Implementation

The shell is rendered by `dashboard.js` into `#view-root`. Routes are hash-based: `#/overview`, `#/journeys`, `#/planning`, `#/reports`, `#/documents`, `#/messages`, and `#/settings`. The sidebar updates the current route; the top bar updates the view title and search placeholder.

`client-data.js` provides the deterministic data. `charts.js` builds the SVG charts. Preferences are persisted in localStorage. Form successes use inline status text and a toast; both clear after roughly 3 seconds.

Responsive notes

Below 900px the sidebar becomes a drawer with a backdrop and focus management. Tables switch to stacked rows with data labels. The top bar keeps search and the account menu. Settings panels stack in a single column. Below 520px the layout tightens spacing and hides non-essential metadata.

> **REVIEW NOTE**
> All data is local mock data. Export actions create real CSV files from that data; account and deletion actions are simulations and never clear the session.

CONSULTANT DASHBOARD

The delivery side of the same system.

consultant-dashboard.html
css/consultant-dashboard.css
js/consultant-dashboard.js

The consultant workspace is a role-themed version of the same shell. It adds a client scope control, portfolio views, a rate card, an inbox with internal notes, and a deactivation flow.

Views

View	Content
Overview	Portfolio health, utilization, alerts, upcoming reviews.
Clients	Client list, health, scope action, detail drawer.
Engagements	Active work, milestones, status, value.
Capacity	Team allocation, bench, weekly sessions.
Revenue	Actual vs forecast, range switch, CSV export.
Deliverables	Review queue, due dates, status.
Inbox	Threads grouped by client, internal notes, composer.
Settings	Profile, rate card, preferences, deactivate.

Role theme

The consultant page adds `role-consultant` to the body. The theme layer swaps the gold accent for teal (`#7CC0B1`), changes the chart lead colour, and updates active navigation, badges, and primary buttons. Layout and typography remain the same.

Settings and scope

- Profile: name, role label, timezone, weekly availability, capacity target, currency.
- Rate card: hourly rates per role, blended rate, validation, save status.
- Preferences: digest, risk alerts, schedule alerts, compact density.
- Deactivate: email confirmation, simulated result, 3-second reset.
- Client scope: a top-bar select that filters every view and is stored in the URL hash.

Messages support a client message or an internal note. The composer switches mode with a radio group; internal notes are visually marked and never presented as client-visible.

Responsive notes

The sidebar becomes a drawer below 900px. Wide tables collapse into stacked rows. The scope control and search remain in the top bar; secondary actions move into the account menu. Settings panels stack. The rate card and chart panels reflow rather than scroll horizontally.

> **REVIEW NOTE**
> The consultant theme is deliberately scoped with a body class, not a separate stylesheet fork. If a third role is added, copy this pattern rather than duplicating the whole dashboard CSS.

RESPONSIVENESS

Designed from 320px outward

The site is built mobile-first in practice, even though the CSS uses a mix of max-width and min-width queries. The narrow layouts are not an afterthought: the hero, process timeline, story gallery, and dashboards all have specific small-screen rules.

Global breakpoints

Width	What changes
≤1100px	Hero columns tighten; footer grid simplifies; nav gaps reduce.
≤900px	Mobile navigation; dashboard sidebar becomes a drawer; several grids collapse.
≤767px	Single-column hero; story, process, and FAQ layouts stack; type scales reduce.
≤520px	Hero stat cards leave the image and stack below it; visual index hidden.
≤430px	Tighter gutters, smaller hero type, story aside hidden, compact footer.
≤420px	Final small-screen spacing and text adjustments.

Page-specific breakpoints

Area	Key widths
About	560, 768, 900, 1000, 1100
Services	560, 768, 900, 1100
Journeys	560, 768, 900, 1100
Planning	560, 768, 900, 1100
Contact	560, 768, 900, 1100
Auth	599, 600, 900
Client dashboard	700, 900, 1150, 1200
Consultant dashboard	400, 520, 700, 900, 1150, 1250

Small-screen fixes worth keeping

- **Hero:** at 520px and below the two stat cards become normal flow items under the image. The image is full width with a 1.15 ratio; the visual index is hidden.
- **Services process:** the diagonal path becomes a vertical rail. The 32px number circle sits in a 44px gutter, leaving a 12px gap before the heading.
- **Story gallery:** the “Stackly field notes” aside sits in the empty left cell of the second gallery row, not on top of the small caption.
- **Story intro:** the intro scrolls with the page rather than sticking, so the eyebrow cannot be trapped under the fixed header.
- **Dashboards:** the sidebar becomes a drawer; tables use stacked rows with data labels; settings panels become one column.

Test matrix

Width	Check
320	No horizontal scroll; hero image visible; process headings clear; forms usable.
375 / 390	Mobile nav; stacked cards; readable type; tap targets.
430 / 520	Hero card handoff; footer wrap; grid changes.
768	Tablet grids; process layout; dashboard drawer.
900 / 1024	Desktop navigation boundary; two-column layouts.
1280 / 1440	Full composition; sticky-free story intro; footer social row.

QA NOTE

Always test at 320px before signing off a layout change. Most of the regressions in this project appeared at the narrowest width, not on desktop.

DEPLOYMENT AND CONCLUSION

GitHub Pages, no build step

The site is deployed directly from the repository. GitHub Pages serves the root of the `main` branch at the project URL. There is no server, pipeline, environment file, or generated output to maintain.

Deployment facts

Item	Value
Repository	`Manjunath0073/Stackly-Adventure-Tourism`
Branch	`main`
Source	Repository root
Live URL	`manjunath0073.github.io/Stackly-Adventure-Tourism/`
Build step	None
Error page	`404.html`
Path rule	All site links and assets use relative paths

Release routine

1. Review the working tree and stage only the intended files.
2. Commit with a short, specific message.
3. Push to `origin main`.
4. Wait for GitHub Pages to rebuild the site.
5. Open the live URL and hard-refresh if the previous CSS is cached.

Path rules

Because the site lives under a repository subpath, root-relative paths such as `/css/style.css` break. Use `css/style.css`, `assets/...`, and page links such as `contact.html`. CSS image URLs use `../assets/...` from inside `css/`.

Verification checklist

Check	Expected
Home page	Styles, images, hero, carousel, FAQ, newsletter load.
Navigation	Header and footer links open real pages.
Content guard	Section CTAs land on the styled 404 page.
Auth	Login and sign-up validate and reach a dashboard.
Dashboards	Sidebar, settings, messages, charts, CSV export work.
Mobile	320px layout has no overlap or horizontal scroll.
404	Direct and guarded visits render the error page.

Troubleshooting

Symptom	Likely cause
Unstyled live page	Root-relative asset path or stale Pages build.
Old CSS after push	Browser cache; hard-refresh or test in a private window.
404 on a real page	Missing `.html` in a link or wrong case in a file name.
Dashboard redirects to login	No valid session or the wrong role is stored.

Conclusion

The project is a complete static front end: a public site, a working account flow, two role-based workspace prototypes, a custom error page, and a deployment path that needs only Git and GitHub Pages. Its main strength is the consistency between the editorial public pages and the denser workspace UI; its main constraint is that all data and account actions are local simulations.

Before production use, connect the forms and auth screens to a real API, replace the mock data modules, add server-side authorization, and decide how content will be edited. The current structure is intentionally simple enough to make those replacements without rewriting the presentation layer.

> **End of document.** Stackly Tourism technical documentation, version 1.0.